```
from databricks.vector_search.client import VectorSearchClient

vsc_endpoint_name = "ml_action_vs"
vsc = VectorSearchClient()

if vsc_endpoint_name not in [e['name'] for e in vsc.list_endpoints()['endpoints']]:
  vsc.create_endpoint(name=vsc_endpoint_name, endpoint_type="STANDARD")
  print(f"Endpoint named {vsc_endpoint_name} is in creation, wait a moment.")

print(f"Endpoint named {vsc_endpoint_name} is ready.")
```

Figure 4.25 – Creating the Databricks VS endpoint for the RAG chatbot

As a reminder, we will ingest the source table we prepared in *Chapter 3* into the index for retrieval search within the chatbot application.

```
display(spark.read.table(f"{catalog}.{database_name}.pdf_documentation_text"))
```

Figure 4.26 – Reading our source table containing text from the documentation

We import functions from the project's `utils` folder, `mlia_utils.rag_funcs`. The index only needs to be created once. Going forward, we will only write into it or read from it. We use an `if/else` clause to check whether an index with the name `catalog.database.docs_vsc_idx_cont` exists or not. If it exists, we just synchronize our `catalog.database.pdf_documentation_text` source table to it.

```
from databricks.sdk import WorkspaceClient
from mlia_utils.rag_funcs import *
# import databricks.sdk.service.catalog as c

# The table we'd like to index
source_table_fullname = f"{catalog}.{database_name}.pdf_documentation_text"
# Where we want to store our index
vs_index_fullname = f"{catalog}.{database_name}.docs_vsc_idx_cont"

if not index_exists(vsc, vsc_endpoint_name, vs_index_fullname):
  print(f"Creating index {vs_index_fullname} on endpoint {vsc_endpoint_name}...")
  vsc.create_delta_sync_index(
    endpoint_name=vsc_endpoint_name,
    index_name=vs_index_fullname,
    source_table_name=source_table_fullname,
    pipeline_type="TRIGGERED", # TRIGGERED or CONTINUOUS
    primary_key="id",
    embedding_dimension=1024, # Match your model embedding size (bge)
    embedding_vector_column="embedding"
  )
else:
  # Trigger a sync to update our vs content with the new data saved in the table
  vsc.get_index(vsc_endpoint_name, vs_index_fullname).sync()
```

Figure 4.27 – Check whether the index exists and create it if needed